LEC 01 DevSecOps & Shift-Left | LEC 02 Secure Coding & Habits | LEC 03 12 Principles & STRIDE | LEC 04 Infra, Cloud & Network | LEC 05 Docker, K8s & Security | LEC 06 CI/CD, SAST/DAST/SCA | LEC 07 SIEM, Logs & IR 6-Steps | LEC 08 GRC, GDPR & HIPAA | LEC 09 Serverless & Wallet DoS

1. DevSecOps Foundations & Shift-Left — LEC 01

DevSecOps: Culture, automation, and platform design integrating security as a **shared responsibility** throughout every stage of the software delivery lifecycle (Plan to Monitor).

TRADITIONAL SDLC VS DEVOPS VS DEVSECOPS

Model	Focus	Security Timing & Weakness
Waterfall	Siloed, sequential	Late pre-release gate; 10–100× fix cost
DevOps	Dev+Ops velocity	Speed prioritized; security bypassed
DevSecOps	Dev+Sec+Ops	**Shift-Left**; continuous, automated security

FOUR CENTRAL DEVSECOPS PRINCIPLES

- **Shared Responsibility:** Security owned by all (Dev, QA, Ops, Sec), not just a siloed security team.
- **Security Automation:** Automated checks (SAST, SCA, secrets, DAST) embedded in CI/CD pipelines.
- **Continuous Security:** Security applied iteratively at every single release cycle and stage.
- **Continuous Monitoring:** 24/7 observability of logs, telemetry, configs, and runtime behaviors.

SHIFT-LEFT SECURITY RATIONALE

Reqs ($1×) → Design ($5×) → Code ($10×) → Test ($15×) → Prod ($100×)

Why Shift Left? Flaws caught early are orders of magnitude cheaper, reduce release delays, empower developers, and prevent systemic production incidents.

SECURITY INTEGRATED PER SDLC PHASE

- **Requirements:** Abuse cases, security user stories, compliance, data classification.
- **Design:** Architectural threat modeling, trust boundaries, secure patterns.
- **Coding:** Secure coding standards, IDE linters, peer review, input validation.
- **Testing:** Automated SAST, SCA, secrets detection, staging DAST.
- **Deployment:** Hardened IaC, immutable images, automated security gates.
- **Operations:** Continuous monitoring, SIEM, IDS/IPS, patch management, IR.

2. Secure Coding Practices — LEC 02

Secure Coding: Proactive software development preventing CIA triad compromise and OWASP Top 10 vulnerabilities.

4 CORE SECURE CODING HABITS

- **1. Input Validation:** Treat ALL external input as untrusted. Use **Allow-list** (regex, type, range, length) over block-lists. *Canonicalize/normalize first → validate → use.*
- **2. Parameterized Queries:** Primary defense against **SQLi**. Separates executable SQL command from untrusted data parameters.
- **3. Output Encoding:** Primary defense against **XSS**. Context-aware sanitization (HTML body, attributes, JS, URLs). E.g., `<` → `<`.
- **4. Least Privilege:** Services & DB users run with bare minimum permissions; restricts attacker **blast radius**.

SQLI VULNERABLE VS SECURE EXAMPLE

```
# INSECURE (Concatenation): "SELECT * FROM users WHERE
name='" + user + "'"
# SECURE (Parameterized): cursor.execute("SELECT *
FROM users WHERE name=?", (user,))
```

2. AuthN, AuthZ & Secure Logging — LEC 02

AUTHENTICATION VS AUTHORIZATION

Dimension	Authentication (AuthN)	Authorization (AuthZ)
Question	*Who are you?*	*What can you access?*
Control	Password, MFA, SSO, Token	RBAC, ABAC, ACL, Scopes
Defense	Argon2id, bcrypt, rate-limit	Default deny; check every request (prevents IDOR)

- **Password Hashing:** Use modern memory-hard functions: **Argon2id**, **bcrypt**, **PBKDF2**. Never use MD5, SHA-1, or plain SHA-256.
- **Logging Rules:** Log AuthN failures, privilege escalation, admin actions. **NEVER log:** plaintext passwords, session tokens, API keys, full PAN, or PII.
- **OWASP Core Focus:** Injection, Broken Auth, Sensitive Data, XXE, Broken Access Control, Security Misconfiguration, XSS, Insecure Deserialization.

3. 12 Secure Design Principles — LEC 03

Secure Design vs Coding: Design governs system architecture and boundaries; coding governs implementation bugs.

Principle	Core Exam Definition & Application
1. Defense in Depth	Layered controls; if one fails, others prevent breach.
2. Fail Securely	Exceptions/errors default to safe, closed state (no bypass).
3. Least Privilege	Minimum permissions needed for minimum required time.
4. Fail-Safe Defaults	Default to **DENY**; explicitly grant permissions.
5. Secure Defaults	Out-of-box configs are secure (ports closed, TLS on).
6. Minimize Surface	Disable unnecessary ports, protocols, services, and APIs.
7. Separation of Duties	Critical actions require 2+ distinct roles/approvals.
8. Complete Mediation	Authorize *every single access* to every object; no cached checks.
9. Economy of Mechanism	Keep security design small, simple, and verifiable.
10. Open Design	Security relies on secret keys, never design secrecy (Kerckhoffs).
11. Least Common Mech.	Minimize shared channels/resources across tenants.
12. Psych. Acceptability	Security must be usable; users won't circumvent it.

3. Threat Modeling & STRIDE — LEC 03

Threat Modeling: Structured technique to identify assets, threats, and mitigations before system construction.

1. Assets → 2. DFD / Boundaries → 3. STRIDE → 4. Mitigate

Trust Boundary: Border where data/control transitions between privilege levels (e.g., Internet → API → DB).

STRIDE THREAT CLASSIFICATION TABLE

Threat	Violates	Exam Definition & Mitigation
S – Spoofing	Authenticity	Impersonating user/process. → *MFA, TLS, AuthN*
T – Tampering	Integrity	Unauthorized modification. → *Hashes, HMAC, TLS*
R – Repudiation	Non-repudiation	Denying an action. → *Tamper-proof audit logs*
I – Info Disclosure	Confidentiality	Data leakage/exposure. → *AES-256, TLS 1.3, ACLs*
D – Denial of Svc	Availability	Resource exhaustion. → *Rate limit, DDoS shield*
E – Elev. of Priv.	Authorization	Gaining higher rights. → *RBAC, least privilege*

RISK ASSESSMENT & 4 TREATMENTS

Formula: $\text{Risk} = \text{Likelihood} \times \text{Impact}$

- **Mitigate:** Apply security control to lower risk (e.g., add WAF).
- **Avoid:** Eliminate risky feature/operation entirely.
- **Transfer:** Shift risk to 3rd party (Cyber insurance, SaaS SLA).
- **Accept:** Acknowledge residual risk with executive approval.

3. Security Design Patterns — LEC 03

- **Secure Gateway:** Single entry point enforcing TLS, AuthN, rate limiting, and request inspection before routing internally.
- **Layered Defense Pattern:** Perimeter Firewall → WAF → DMZ API Gateway → mTLS Microservices → Encrypted DB with isolated credentials.

✍ INTENTIONAL BLANK SPACE FOR HANDWRITTEN EXAM NOTES & FORMULAS (SIDE A)

PRESERVED WRITING SPACE

4. Infrastructure & Cloud Security LEC 04

Cloud Shared Responsibility Model: CSP manages **Security OF the Cloud** (hardware, hypervisor, facilities). Customer manages **Security IN the Cloud** (data, IAM, OS, apps, firewalls).

Model	CSP Manages	Customer Manages
IaaS	Hardware, Virtualization	Guest OS, Apps, Data, IAM, Config
PaaS	Hardware, OS, Runtime	App Code, Data, User IAM
SaaS	Entire App Stack & Infra	Data Governance, User Access

AWS VS AZURE SECURITY SERVICES

Capability	AWS	Azure
Identity / IAM	AWS IAM	Microsoft Entra ID
Activity Audit	CloudTrail	Activity Log
Monitoring	CloudWatch	Azure Monitor
Posture / Threat	GuardDuty / Sec Hub	Defender for Cloud
Secrets & Keys	KMS / Secrets Mgr	Key Vault
Network Firewall	Sec Groups & NACLs	NSGs & Azure Firewall

DATA RESILIENCE & NETWORK CONTROLS

- **RTO:** Max acceptable downtime duration before restore.
- **RPO:** Max acceptable data loss time window (age of data).
- **IDS vs IPS:** *IDS* = passive, out-of-band alerts; *IPS* = active, inline blocking of malicious traffic.
- **WAF:** Layer 7 application inspection (SQLi, XSS filtering).
- **IAM Models: RBAC** (roles), **ABAC** (attributes/context), **DAC** (owner controls), **MAC** (fixed security labels).
- **Hardening & IaC:** Enforce CIS baselines, disable unused ports, prevent configuration drift via declarative Terraform/Ansible.

5. Containers & Kubernetes LEC 05

VIRTUAL MACHINES (VM) VS CONTAINERS

Virtual Machines (VM)	Containers (Docker)
Hypervisor + Full Guest OS	Shares Host OS Kernel (namespaces/cgroups)
Heavy overhead, minutes boot	Lightweight, millisecond startup
Strong hardware isolation	Process isolation; container escape risk

DOCKER & CONTAINER HARDENING

- **Non-Root:** Use `USER nonroot`; avoid UID 0. Drop caps: `--cap-drop=ALL`.
- **No Privileged:** Never pass `--privileged`. Never mount `/var/run/docker.sock` inside containers (prevents escape).
- **Secrets:** Never hardcode in Dockerfile (`ENV`/`ARG`); use runtime vaults.
- **Runtime:** Read-only rootfs (`--read-only`), seccomp, AppArmor.

KUBERNETES (K8S) ARCHITECTURE & SECURITY

- **Control Plane:** API Server (mTLS/RBAC), etcd (encrypted at rest), Scheduler, Controller Mgr.
- **Worker Node:** kubelet, kube-proxy, container runtime, Pods.
- **Security Controls:** K8s RBAC (Roles/Bindings), `NetworkPolicy` (default deny pod ingress/egress), Pod Security Standards (PSS/PSA).

6. Secure CI/CD & Testing LEC 06

CI/CD Pipeline: CI (frequent code integration & automated tests) → Continuous Delivery (deployable build + manual gate) → Continuous Deployment (auto to prod).

SECURITY TESTING CLASS COMPARISON

Class	Analyzes	App Running?	Primary Detection
SAST	Source code / AST	No (White-box)	Insecure code, SQLi, syntax flaws
DAST	Running HTTP API	Yes (Black-box)	Runtime XSS, auth bypass, headers
SCA	Dependencies	No	Known CVEs, outdated pkgs, licenses
Secrets	Git commits/history	No	Committed tokens, API keys, passwords
IaC	Terraform/K8s files	No	Misconfigured ports, open SGs, plain S3

SECURITY GATES & SUPPLY CHAIN

- **Security Gates:** Automated thresholds stopping build on High/Critical CVEs or exposed secrets.
- **Supply Chain Security: SBOM** (CycloneDX/SPDX), artifact signing (**Cosign/Sigstore**), pinned dependencies, isolated runners.

7. Monitoring & Incident Response LEC 07

Continuous Security Monitoring (CSM): Real-time observability of telemetry. Feedback loops back to SDLC Plan.

- **Log Management:** Sync UTC clocks via NTP. Must log: timestamp, src IP, user, action, result. *Mask secrets/PII*; use tamper-evident storage.
- **SIEM:** Aggregates, normalizes, correlates multi-source logs. *False Positive* = benign flagged bad; *False Negative* = breach missed.

INCIDENT RESPONSE (IR) 6-PHASE LIFECYCLE

1. Prep → 2. Detect → 3. Contain → 4. Eradicate → 5. Recover → 6. Lessons

- **Containment:** Short-term (isolate VM, revoke tokens) & long-term.
- **Forensics:** Preserve bit-stream copies (work on copies, not live), verify SHA-256 hashes, maintain strict **Chain of Custody**.

8. Compliance & Governance (GRC) LEC 08

GRC: Governance (oversight/policy) + **Risk** (management) + **Compliance** (legal proof). *Compliance ≠ Security* (compliance is baseline; security is operational defense).

Reg / Standard	Type	Core Scope & Exam Points
GDPR	EU Statutory Law	Personal data. 7 Principles. **72h breach notice** (Art 33). Extraterritorial.
HIPAA	US Statutory Law	**PHI / ePHI**. Privacy Rule, Security Rule (Admin, Physical, Tech), Breach Rule.
ENISA	EU Agency	**Advisory Agency (NOT law!)**. Best practices and cyber threat reports.
ISO 27001	Standard	ISMS specification; auditable certification.

- **Control Types:** *Administrative* (policies, training), *Technical* (MFA, encryption), *Physical* (locks, guards).
- **Audit vs Assessment:** Audit = formal compliance evaluation; Assessment = internal risk & gap discovery.

9. Secure Serverless & Cloud-Native LEC 09

Serverless / FaaS: Event-driven, auto-scaling, ephemeral execution (AWS Lambda, Azure Functions). *Servers exist, but customer does not manage OS/servers!*

SHARED RESPONSIBILITY IN SERVERLESS

- **CSP Manages:** Physical infra, host OS, container runtime, auto-scaling, platform availability.
- **Customer Manages:** Function code, IAM execution roles, API gateway auth, secrets, data encryption, third-party libraries.

SERVERLESS THREATS & MITIGATIONS

- **Excessive Permissions:** Function compromise spreads. → *Enforce scoped per-function IAM least privilege.*
- **Insecure APIs:** Public invocation. → *API Gateway auth (JWT/OAuth2), TLS, WAF.*
- **Secrets Leak:** Plaintext in env vars. → *AWS Secrets Mgr / Key Vault.*
- **Event Injection:** Malicious queue/payload. → *Validate all events.*
- **Denial of Wallet (DoW):** Flood of event triggers causes massive financial billing spikes. → *API throttling, concurrency limits, budget kill-switches & billing alarms.*

✍ INTENTIONAL BLANK SPACE FOR HANDWRITTEN EXAM NOTES & FORMULAS (SIDE B) — PRESERVED WRITING SPACE